Lab Final

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

1 Lab Final

1. Write a query that returns customer_id, first_name, last_name, and the length of their email address. Only include customers whose email length is greater than 25 characters. Order by email length in descending order.

```
SELECT customer_id,
       first_name,
       last_name,
       LENGTH(email) AS email_length
FROM customer
WHERE LENGTH(email) > 25
ORDER BY email_length DESC;
```

2. Identify customers who have made at least 20 payments with an average payment amount above $5.00. Show the customer's first and last name, the total number of payments made, and their average payment amount.

```
SELECT
    c.first_name,
    c.last_name,
    COUNT(p.payment_id) AS total_payments,
    ROUND(AVG(p.amount), 2) AS average_payment
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(p.payment_id) >= 20
   AND AVG(p.amount) > 5.00
ORDER BY average_payment DESC;
```

3. Write a query that lists the actor_id, first_name, last_name, and the total number of films for each actor. Only include actors who have appeared in more films than the overall average number of films per actor. Order the results by film count in descending order.

```
SELECT a.actor_id,
       a.first_name,
       a.last_name,
       COUNT(fa.film_id) AS film_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(fa.film_id) > (
    SELECT AVG(actor_counts.f_count)
    FROM (
        SELECT COUNT(film_id) AS f_count
        FROM film_actor
        GROUP BY actor_id
    ) AS actor_counts
)
ORDER BY film_count DESC;
```

4. Write a query that lists customers whose total spending is greater than the average spending of customers within the same store. Show store_id, customer_id, and total_spent.

```sql
SELECT c.store_id,
       c.customer_id,
       SUM(p.amount) AS total_spent
FROM customer c
JOIN payment p ON p.customer_id = c.customer_id
GROUP BY c.store_id, c.customer_id
HAVING SUM(p.amount) >
       (
           SELECT AVG(store_total)
           FROM (
               SELECT SUM(p2.amount) AS store_total
               FROM customer c2
               JOIN payment p2 ON p2.customer_id = c2.customer_id
               WHERE c2.store_id = c.store_id
               GROUP BY c2.customer_id
           ) sub
       )
ORDER BY c.store_id, total_spent DESC;
```

5. Write an anonymous PL/pgSQL block that calculates and sequentially prints the Fibonacci sequence up to the Nth number. The sequence starts with 0 and 1, and every subsequent number is the sum of the previous two. The first 10 numbers of the Fibonacci sequence are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

```sql
DO $$
DECLARE
    n INT := 10;
    a INT := 0;
    b INT := 1;
    temp INT;
BEGIN
    RAISE NOTICE 'Printing the first % Fibonacci numbers:', n;

    FOR i IN 1..n LOOP
        RAISE NOTICE '%', a;

        -- Calculate the next number in the sequence
        temp := a + b;

        -- Shift the values forward for the next iteration
        a := b;
        b := temp;
    END LOOP;
END $$;
```

6. Write an anonymous PL/pgSQL block that prints all film title, description and description length where the film's description mentions a "Moose" and a "Robot" or a "Moose" and a "Dentist" anywhere in the sentence, and the description length is strictly greater than 20 characters. Order the output by description length in descending order.

```sql
DO $$
DECLARE
    rec RECORD;
BEGIN
```

```
    FOR rec IN
        SELECT film_id, title, length, description
        FROM film
        WHERE LENGTH(description) > 20
          AND description LIKE '%Moose%'
          AND (description LIKE '%Robot%' OR description LIKE '%Dentist%')
        ORDER BY length DESC, film_id
    LOOP
        RAISE NOTICE 'Film %: % (Length: %) | Desc: %',
            rec.film_id, rec.title, rec.length, rec.description;
    END LOOP;
END $$;
```

7. Write a PL/pgSQL function to find the best-selling film of a specific category. The category name will be the input parameter, and the function should return the title of the film that generated the highest total revenue

```
CREATE OR REPLACE FUNCTION get_best_selling_film_by_category(p_category_name
    VARCHAR)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    v_film_title VARCHAR(255);
BEGIN
    SELECT f.title INTO v_film_title
    FROM category c
    JOIN film_category fc ON c.category_id = fc.category_id
    JOIN film f ON fc.film_id = f.film_id
    JOIN inventory i ON f.film_id = i.film_id
    JOIN rental r ON i.inventory_id = r.inventory_id
    JOIN payment p ON r.rental_id = p.rental_id
    WHERE c.name = p_category_name
    GROUP BY f.film_id, f.title
    ORDER BY SUM(p.amount) DESC
    LIMIT 1;

    RETURN v_film_title;
END;
$$;
SELECT get_best_selling_film_by_category('Action') AS best_selling_film;
```

8. Create a stored procedure inventory_availability_summary to summarize a film's availability in a specific store. It must count total copies, count available copies, and update an input variable with the availability percentage.

```
CREATE OR REPLACE PROCEDURE inventory_availability_summary(
    p_film_id INT,
    p_store_id INT,
    INOUT p_avail_pct NUMERIC,
    OUT o_total_copies INT,
    OUT o_available_copies INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_rented_copies INT;
```

```
BEGIN
    SELECT COUNT(*) INTO o_total_copies
    FROM inventory
    WHERE film_id = p_film_id AND store_id = p_store_id;

    SELECT COUNT(DISTINCT r.inventory_id) INTO v_rented_copies
    FROM rental r
    JOIN inventory i ON i.inventory_id = r.inventory_id
    WHERE i.film_id = p_film_id AND i.store_id = p_store_id AND r.return_date IS
     NULL;

    o_available_copies := o_total_copies - v_rented_copies;

    IF o_total_copies = 0 THEN
        p_avail_pct := 0;
    ELSE
        p_avail_pct := ROUND((o_available_copies::NUMERIC * 100.0) /
    o_total_copies, 2);
    END IF;
END;
$$;

DO $$
DECLARE
    -- Initialize the INOUT parameter
    v_availability_percentage NUMERIC := 0.00;
    v_total INT;
    v_available INT;
BEGIN
    CALL inventory_availability_summary(
        586,                          -- p_film_id (IN)
        1,                           -- p_store_id (IN)
        v_availability_percentage,   -- p_avail_pct (INOUT)
        v_total,                     -- o_total_copies (OUT)
        v_available                  -- o_available_copies (OUT)
    );

    -- Print the returned values
    RAISE NOTICE 'Total Inventory: %', v_total;
    RAISE NOTICE 'Available for Rent: %', v_available;
    RAISE NOTICE 'Availability: %%%', v_availability_percentage;
END;
$$;
```

9. Write a PL/pgSQL function that takes two store IDs, compares their customers' average lifetime spending, and returns a message stating which store is more profitable.

```
CREATE OR REPLACE FUNCTION compare_store_profitability(store_a_id INT,
    store_b_id INT)
RETURNS VARCHAR AS $$
DECLARE
    store_a_avg DECIMAL(10,2);
    store_b_avg DECIMAL(10,2);
    result_msg VARCHAR(255);
BEGIN
    SELECT AVG(customer_total) INTO store_a_avg
```

```
    FROM (
        SELECT c.customer_id, SUM(p.amount) AS customer_total
        FROM customer c
        JOIN payment p ON c.customer_id = p.customer_id
        WHERE c.store_id = store_a_id
        GROUP BY c.customer_id
    ) a_totals;

    SELECT AVG(customer_total) INTO store_b_avg
    FROM (
        SELECT c.customer_id, SUM(p.amount) AS customer_total
        FROM customer c
        JOIN payment p ON c.customer_id = p.customer_id
        WHERE c.store_id = store_b_id
        GROUP BY c.customer_id
    ) b_totals;

    store_a_avg := COALESCE(store_a_avg, 0);
    store_b_avg := COALESCE(store_b_avg, 0);

    IF store_a_avg > store_b_avg THEN
        result_msg := 'Store ' || store_a_id || ' is stronger (Avg: $' || ROUND(
    store_a_avg, 2) || ' vs $' || ROUND(store_b_avg, 2) || ')';
    ELSIF store_b_avg > store_a_avg THEN
        result_msg := 'Store ' || store_b_id || ' is stronger (Avg: $' || ROUND(
    store_b_avg, 2) || ' vs $' || ROUND(store_a_avg, 2) || ')';
    ELSE
        result_msg := 'Both stores have equal profitability (Avg: $' || ROUND(
    store_a_avg, 2) || ')';
    END IF;

    RETURN result_msg;
END;
$$ LANGUAGE plpgsql;
```

10. Create a function that takes a customer ID and returns the name of the most-rented category for that customer. If the customer has no rentals, return NULL.

```
CREATE OR REPLACE FUNCTION top_category_name(p_customer_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_name TEXT;
BEGIN
    SELECT c.name
    INTO v_name
    FROM rental r
    JOIN inventory i ON i.inventory_id = r.inventory_id
    JOIN film_category fc ON fc.film_id = i.film_id
    JOIN category c ON c.category_id = fc.category_id
    WHERE r.customer_id = p_customer_id
    GROUP BY c.name, c.category_id
    ORDER BY COUNT(*) DESC, c.category_id
    LIMIT 1;

    RETURN v_name; -- NULL if none
END;
```

```
$$;

-- Invocation (example)
SELECT top_category_name(1);
```

2 ER Diagram


actor
actor_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
last_update DATETIME *
language
language_id INT * pk
name CHAR(20) *
last_update DATETIME *
film
film_id INT * pk
title VARCHAR(255) *
description TEXT
release_year YEAR
language_id INT *
rental_duration SMALLINT *
rental_rate DECIMAL(4,2) *
length SMALLINT
replacement_cost DECIMAL(5,2) *
rating film_rating_enum
last_update DATETIME *
special_features TEXT
fulltext_col TEXT *
category
category_id INT * pk
name VARCHAR(25) *
last_update DATETIME *
country
country_id INT * pk
country VARCHAR(50) *
last_update DATETIME *
city
city_id INT * pk
city VARCHAR(50) *
country_id INT *
last_update DATETIME *
address
address_id INT * pk
address VARCHAR(50) *
address2 VARCHAR(50)
district VARCHAR(20) *
city_id INT *
postal_code VARCHAR(10)
phone VARCHAR(20) *
last_update DATETIME *
film_actor
actor_id INT *
film_id INT *
last_update DATETIME *
inventory
inventory_id INT * pk
film_id INT *
store_id INT *
last_update DATETIME *
film_category
film_id INT *
category_id INT *
last_update DATETIME *
customer
customer_id INT * pk
store_id INT *
first_name VARCHAR(45) *
last_name VARCHAR(45) *
email VARCHAR(50)
address_id INT *
activebool BOOLEAN *
create_date DATE *
last_update DATETIME
active INT
staff
staff_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
address_id INT *
email VARCHAR(50)
store_id INT *
active BOOLEAN *
username VARCHAR(16) *
password VARCHAR(40)
last_update DATETIME *
picture BLOB
rental
rental_id INT * pk
rental_date DATETIME *
inventory_id INT *
customer_id INT *
return_date DATETIME
staff_id INT *
last_update DATETIME *
store
store_id INT * pk
manager_staff_id INT *
address_id INT *
last_update DATETIME *
payment
payment_id INT * pk
customer_id INT *
staff_id INT *
rental_id INT
amount DECIMAL(5,2) *
payment_date DATETIME *